

Paluwagan

On-chain Rotating Savings & Credit Association (ROSCA)
on the Bitcoin Cash Chipnet testnet

Project Report — Full Project Documentation

Generated August 2026

1. Overview

Paluwagan digitises the traditional Filipino paluwagan — a rotating savings and credit association — as a web application backed by smart contracts on the Bitcoin Cash Chipnet testnet. A group of members commits to pay a fixed contribution each round; every round, one member receives the entire pot. Contributions, rounds, and payouts are enforced on-chain by a CashScript contract (RoundPot) and orchestrated from a Laravel backend.

The application provides a manager dashboard (batch list, total value locked, per-batch round progress), a members workspace (contribution tracking, payout order, round controls), a live transaction history pulled from the Chipnet block explorer showing every member contribution paid into the pot, and a wallet-based member portal where a participant can view every circle they joined or created. Every BCH amount is also shown with a live Philippine Peso (₱) equivalent. Trust is replaced by verifiable on-chain records.

2. Key Features

- ROSCA circles — create batches with schedule (monthly / weekly / daily), contribution amount, and rotation policy.
- Payout rotation — fixed pays by membership position; random shuffles the payout order once per cycle so every member gets exactly one pot.
- On-chain enforcement — CashScript RoundPot contract locks contributions, lets the round recipient claim within a block window, and lets the organizer reclaim via `timeout()` if unclaimed.
- Member tracking — wallet address per member (name optional), contribution history, savings balance, status (Active / Released / Slashed), and auto-pay flag.
- Contributions log — live transaction history for each pot, sourced from `chipnet.bchexplorer.info`, showing date, wallet address, member name, amount, and txid.
- Manager overview — total value locked across all batches, contribution progress, and round status per batch.
- Member self-service portal — register your BCH wallet once per session (no login required); see every circle you participate in and every circle you created.
- Peso pricing — each BCH amount is accompanied by its live Philippine Peso (₱) equivalent, priced from the CoinGecko feed with a cached fallback rate.
- Full auth stack — Fortify-backed registration, login, email verification, two-factor authentication, and passkeys.

3. Technology Stack

Layer	Technology	Role
Backend	Laravel 13 (PHP 8.5)	API, domain logic, on-chain orchestration
Frontend	Inertia.js v3 + React 19 + Tailwind CSS 4	Server-driven SPA without API boilerplate
Routing	Laravel Wayfinder	Type-safe route helpers generated for the frontend
Auth	Laravel Fortify	Auth backend (2FA, passkeys, verification)
Smart contracts	CashScript (sBTC)	Per-round pot contract + Node worker simulator
Block explorer	Chipnet (<code>bchexplorer.info</code>)	Live on-chain transaction data (Fulcrum API)
Fiat pricing	CoinGecko API	Live BCH → PHP exchange rate (cached)
Testing	Pest 5 / PHPUnit	Feature & unit coverage

Note: the contracts/ package (TypeScript) is compiled with CashScript and executed by RoundService via a Node worker process (`worker.ts`), which simulates and returns the contract address, payout txid, and phase for each round.

4. Architecture

A classic Laravel + Inertia monolith. Server-side controllers render Inertia pages; the React frontend is a client-side SPA driven by server data. On-chain state lives in the `contracts/` package; the Laravel side records round results in the ledger and pulls live contribution data from the Chipnet explorer API. The BCH → PHP rate is resolved server-side, cached, and shared to every Inertia page as a global currency prop.

Component	Path	Responsibility
Web routes	<code>routes/web.php</code>	Batches, members, member portal, dashboard, settings
Controllers	<code>app/Http/Controllers</code>	Batch & member HTTP handlers, member portal
Services	<code>app/Services</code>	Round orchestration, explorer + fiat integration
Models	<code>app/Models</code>	Batch, Member, BatchMember, BatchContribution
Console	<code>app/Console/Commands</code>	<code>round:advance</code> , <code>round:expire</code>
Contracts	<code>contracts/src</code>	RoundPot CashScript + worker
Pages	<code>resources/js/pages</code>	Inertia + React views (manager, member, home)

5. Domain Model

Schema

Table	Key columns	Purpose
<code>users</code>	email, password, two-factor, passkeys	Organizer accounts (Fortify)
<code>members</code>	name (nullable), wallet	Participants and their chipnet addresses
<code>batches</code>	name, status, schedule, rotation, <code>contribution_sats</code> , <code>rounds_total</code> , <code>rounds_current</code> , <code>contract_address</code> , <code>pot_address</code> , <code>created_by_wallet</code> , <code>last_payout_tx</code>	A savings circle; <code>created_by_wallet</code> links it back to the wallet that started it
<code>batch_members</code>	<code>batch_id</code> , <code>member_id</code> , position, <code>payout_order</code> , status, <code>auto_pay</code>	Membership pivot; unique per batch + member
<code>batch_contributions</code>	<code>batch_member_id</code> , round, <code>amount_sats</code> , <code>tx_id</code>	Per-round ledger; unique per member + round

Relationships

Batch ↔ BatchMember (hasMany) ↔ Member (belongsToMany via pivot). BatchMember → BatchContribution (hasMany); Batch aggregates contributions through hasManyThrough. Deletes cascade from `batch_members` down to `batch_contributions`; members survive batch deletion. Batches also carry a `created_by_wallet` column so the member portal can list the circles a given wallet created.

State Machines

Entity	States	Transitions
Batch	Forming → Active → Completed	<code>created</code> → Active on first round; Completed at <code>rounds_current == rounds_total</code>
BatchMember	Active / Released / Slashed	recipient Released after claiming pot

6. On-Chain Round Flow

RoundPot contract

`contracts/src/round_pot.cash` locks the round's pot (`contributionSats × memberCount`) into a P2SH contract between `startBlock` and `deadline`:

`claim(pk, s)` — the round recipient withdraws the full pot (minus 2000 sats fee) while the round is live.

`timeout(pk, s)` — after the deadline, the organizer reclaims the unclaimed pot, so funds are never stuck.

Round lifecycle

Step	Action	On-chain / ledger effect
1	Create batch + members	Rows in batches, members, batch_members
2	Assign payout order	Fixed = position; random = one shuffle per cycle
3	Advance round	worker returns contractAddress/txid; BatchContribution rows written for every member
4	Recipient claims	Member marked Released; last_payout_tx recorded
5	Round expires (optional)	Organizer reclaims pot via <code>timeout()</code>
6	Contribution log	ChipnetExplorerService pulls pot txs from the Chipnet API

Explorer integration

ChipnetExplorerService queries <https://chipnet.bchexplorer.info/api/bch/address/{pot}/txs> (cursor paginated, 100/page, up to 3 pages), keeps only transactions whose vin sender is a batch member's wallet, maps senders to member names (fallback Member {position}), formats amount and date, and caches the result for 60 seconds. Network failure degrades gracefully to an empty log.

7. Fiat (PHP) Conversion

Every BCH amount in the application is accompanied by a Philippine Peso (₱) equivalent. FiatConversionService fetches the live BCH/PHP rate from the CoinGecko price feed, caches it for ten minutes, and falls back to a configured rate when the feed is unreachable or returns no price. The rate is resolved server-side and shared to every Inertia response as a global currency prop (rate + code + symbol).

Mechanism	Details
Rate source	GET https://api.coingecko.com/api/v3/simple/price?ids=bitcoin-cash&vs_currencies=php
Caching	Rate cached for 600 s in the app cache — one network call per window
Fallback	<code>config('fiat.bch_php_rate')</code> / env <code>BCH_PHP_RATE</code> (default 25,000) used if the feed fails or returns no price
Conversion	$\text{sats} \div 100,000,000 \times \text{rate}$, formatted as PHP currency (₱) with thousands separators
Configuration	<code>config/fiat.php</code> — fallback rate + cache TTL
Display	Reusable Amount component renders the BCH value first with a muted $\approx$ ₱... sub-line; used on manager and member pages

The Amount component reads the shared rate from the page props, so all money displays — total value locked, pot balances, saved / contribution / remaining amounts, target payouts, and the explorer transaction log — show both the BCH token amount and its peso value without any server-side string duplication.

8. HTTP API / Routes

Meth od	URI	Name	Handler
GET	/	home	Inertia page (landing)
GET	/dashboard	dashboard	Inertia page (auth + verified)
GET	/batches	batches.index	BatchController@index
POST	/batches	batches.store	BatchController@store
GET	/batches/{batch}	batches.show	BatchController@show
POST	/batches/{batch}/advance	batches.advance	BatchController@advance
POST	/batches/{batch}/expire	batches.expire	BatchController@expire
POST	/members	members.store	MemberController@store
GET	/member	member.index	MemberPortalController@index (wallet registration)
POST	/member/register	member.register	MemberPortalController@register (binds wallet to session)
GET	/member/batches	member.batch	MemberPortalController@batch (circles the wallet joined)
GET	/member/batches/created	member.created	MemberPortalController@created (circles the wallet created)
POST	/member/forget	member.forget	MemberPortalController@forget (clears session wallet)

Plus Fortify auth routes (login, register, 2FA, passkeys, verification) and settings routes (profile, security) in `routes/settings.php`. Frontend route helpers are generated by Wayfinder (`@/routes/*`) so pages use typed route functions instead of hard-coded URLs.

9. Services & Commands

Class	Location	Responsibility
RoundService	<code>app/Services</code>	<code>advance()/expire()</code> — picks recipient, runs worker, records contributions
ChipnetExplorerService	<code>app/Services</code>	Resolves pot transactions into the contributions log
FiatConversionService	<code>app/Services</code>	Resolves and caches the BCH/PHP rate; converts sats to ₱
RoundAdvance	<code>app/Console/Commands</code>	<code>php artisan round:advance {batch}</code>
RoundExpire	<code>app/Console/Commands</code>	<code>php artisan round:expire {batch}</code>

10. Frontend Pages

Page	Route	Contents
Home	/	Product landing page
Dashboard	/dashboard	Authenticated home
Batches (manager)	/batches	Batch cards with pot, round progress, status; total value locked card; create-batch dialog; wallet selector
Members (manager)	/batches/{batch}	Total balance, on-chain transaction history (Date / Wallet / Name / Txid / Amount), member cards with savings & progress, round controls (advance / expire)
Member portal	/member	Standalone wallet registration screen — no login required
My savings circles	/member/batches	Every circle the wallet joined: saved amount, contribution, target payout, remaining due, contribution table, payout rounds
My batches	/member/batches/created	Every circle the wallet created, with progress, target payout, and contribution
Settings — Profile	/settings/profile	Profile, password, session management
Settings — Security	/settings/security	2FA, passkeys, account deletion
Auth	/login, /register, 2FA challenge, password reset, email verification	Fortify-backed flows

Manager UI highlights

- Total Value Locked — summed from all batches' on-ledger contributions (sats), rendered server-side.
- Transaction History — real pot transactions from the Chipnet explorer: date, wallet address, member name (or Member N), txid, amount.
- Round Controls — advance or expire the current round with server-side validation and flash feedback.
- Member Cards — address copy-to-clipboard, savings balance, contribution progress bar, due/remaining amounts, auto-pay toggle, status badge.
- Peso equivalents — the Amount component appends a ₱... sub-line to every BCH figure.

Member portal highlights

- Wallet identity — a wallet address entered once is stored in the session as the member's identity; no account or password needed.
- My savings circles — grouped by batch, each showing saved / contribution / target payout / remaining plus the per-round contribution ledger and payout schedule.
- My batches — a grid of circles the wallet created, with the same ₱ conversion applied.
- Forget wallet — one click clears the session wallet and returns to registration.
- Shared layout — the sidebar is role-aware: member items (My Savings Circles, My Batches) replace manager items when browsing the /member area.

11. Testing

Pest 5 feature suite (`php artisan test`) covers batches, members, round orchestration, explorer integration, fiat conversion, the member portal, dashboard, and the Fortify auth flows — 80 feature tests. The on-chain explorer service and the fiat service are tested with faked HTTP responses.

Suite	Coverage
BatchMemberControllerTest	Batch listing w/ derived pot & progress, members page, total value locked, member creation

Suite	Coverage
ChipnetExplorerServiceTest	Contribution log resolution, fallback labels, failure handling
FiatConversionServiceTest	Live rate, caching, fallback rate, sats → ₿ math, shared currency prop on responses
MemberPortalTest	Wallet register / forget, redirects, participant listing, member point-of-view payloads, created-only listing
RoundServiceTest	Round advance / expire orchestration
BatchRoscTest	Models, ledger, constraints, cascades
CreateBatchTest / DashboardTest	Batch creation, dashboard
tests/Feature/Auth, tests/Feature/Settings	Fortify + settings flows

Tooling: Laravel Pint (style), Larastan (static analysis), ESLint/Prettier + TypeScript checks (frontend). CI script: `composer ci:check`.

12. Project Structure

Directory	Contents
app/Models	Batch, BatchMember, BatchContribution, Member, User
app/Services	RoundService, ChipnetExplorerService, FiatConversionService
app/Http/Controllers	BatchController, MemberController, MemberPortalController, Settings/*
app/Console/Commands	RoundAdvance, RoundExpire
config/fiat.php	BCH/PHP fallback rate + cache TTL
contracts/src	round_pot.cash, worker.ts, keys.ts, demo.ts, artifacts
database/migrations	users, members, batches, batch_members, batch_contributions, created_by_wallet, Fortify columns
resources/js/pages	Inertia + React pages (manager, member, auth, settings, home)
resources/js/components	amount.tsx (BCH + ₿ display), layout, UI primitives
resources/js/lib	currency.ts (BCH → PHP formatting), utils.ts
tests	Pest feature/unit suites

13. Setup & Run

- `composer install`
- `cp .env.example .env && php artisan key:generate`
- `php artisan migrate --force`
- `npm install && npm run build`
- Contracts: `cd contracts && npm install` (CashScript compilation via `cashc`).
- Dev: `composer run dev` (or `php artisan serve + npm run dev`).
- Single command alternative: `composer setup`.

14. Verification & Quality

- `vendor/bin/pint` — PHP code style.
- `php artisan test` — Pest feature/unit suites.
- `composer ci:check` — lint + static analysis + types + tests in one pass.
- Contracts verified with CashScript tests (`contracts/test`).

15. Future Work

- Wire real wallet funding so contributions are actually broadcast to Chipnet (currently simulated by the worker).
- Push notifications / scheduling for round deadlines (via `schedule`).
- Per-member contribution CSV export and receipts.
- Multi-organizer workspaces and role-based access.
- Rate history / charting for the ₱ conversion and configurable fiat currencies.
- Deploy to the main Bitcoin Cash network after Chipnet validation.